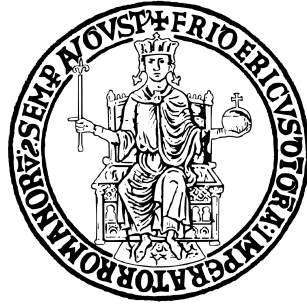




BugBoard26

Progetto di Ingegneria del Software – A.A. 2025/2026

Autori:

Simone Parente Martone - N86004297

Mario Penna - N86003308

Michela Pollio - N86003697

Dipartimento di Ingegneria Elettrica e delle Tecnologie
dell'Informazione (DIETI)
Università degli Studi di Napoli Federico II

Indice

1	Introduzione	3
2	Specifica dei Requisiti Software	4
2.1	Attori del Sistema	4
2.2	Requisiti Non Funzionali	5
2.2.1	Requisiti del Prodotto	5
2.2.2	Requisiti Organizzativi	7
2.3	Modellazione dei Casi d'uso	9
2.4	Casi d'uso Significativi	10
2.4.1	UC1 - Creazione di una issue	10
2.4.2	UC2 - Associazione di etichette personalizzate	15
2.4.3	UC3 – Ordinamento delle Issue	16
3	Definizione dell'architettura software	19
3.1	Introduzione	19
3.2	Architettura Logica	19
3.3	Pattern Architetturale Backend	20
3.4	Stack tecnologico backend	20
3.5	Pattern Architetturale Frontend	21
3.6	Dettaglio Tecnologie Frontend	21
3.7	Architettura Fisica e Deployment	22
3.8	Descrizione dello schema per la persistenza dati	23
4	Definizione dei Design Patterns	24
4.1	Singleton Pattern	24

4.2	Dependency Injection e Inversion of Control (IoC)	24
4.3	Strategy Pattern	25
5	Processo di Sviluppo e CI/CD	26
5.1	Gestione del Versioning	26
5.2	Continuous Integration	26
5.3	Workflow GitHub Actions Implementati	27
5.3.1	Verifica della formattazione del codice	27
5.3.2	Build automatica	27
5.3.3	Analisi statica del codice tramite SonarQube	28
5.4	Continuous Deployment	28
5.5	Gestione della Concorrenza delle Pipeline	29
6	Attività di Testing	30
6.1	Strategia Generale di Testing	30
6.2	Tecniche di Progettazione dei Test	31
6.2.1	Black Box Testing	31
6.2.2	White Box Testing	32
6.3	Architettura dei Test Backend	33
6.3.1	Unit Testing del Service Layer	33
6.4	Test Plan della creazione di una Issue	51
6.4.1	Obiettivi	52
6.4.2	Scope	52
6.4.3	Precondizioni	53
6.4.4	Classi di equivalenza	54
6.4.5	Tracciabilità dei casi di test	55
6.4.6	Criteri di accettazione	56
7	Glossario	57

1 Introduzione

Il presente documento descrive l'analisi e la specifica dei requisiti per il progetto *BugBoard26*, una piattaforma software per la gestione collaborativa di *issue* e *bug* durante il ciclo di sviluppo di un prodotto software.

L'obiettivo principale del sistema è fornire un ambiente:

- **semplice**, per facilitare la segnalazione e la gestione dei problemi;
- **sicuro**, per garantire l'integrità e la riservatezza dei dati;
- **efficiente**, per supportare le attività di classificazione, assegnazione e monitoraggio delle problematiche software.

Il sistema integra funzionalità avanzate come:

- filtri dinamici di ricerca,
- allegati di file,
- esportazione dei dati,
- generazione di report mensili sull'attività del team.

Il documento include:

- la specifica dettagliata dei requisiti software;
- la modellazione dei casi d'uso secondo il formalismo di *Alistair Cockburn* [1];
- la descrizione dell'interazione tra utenti e sistema, supportata da diagrammi e mock-up.

Questo testo costituisce una guida completa per **sviluppatori**, **project manager** e **stakeholder**, fornendo una visione chiara e strutturata delle funzionalità e delle logiche alla base del progetto *BugBoard26*.

2 Specifica dei Requisiti Software

2.1 Attori del Sistema

Considerando il tipo di sistema in sviluppo — una piattaforma per la gestione collaborativa di progetti software — il background tecnico degli utenti sarà piuttosto eterogeneo e sono state individuate tre classi di utenti:

- **Tecnici:** sviluppatori software, tester e altre figure tecniche operative
- **Amministratori:** team leaders, managers e altre figure con responsabilità di coordinamento e supervisione del progetto
- **Esterni:** entità con un interesse diretto o indiretto nelle attività, nelle decisioni o nei risultati di un progetto

Tabella 2.1: Caratterizzazione degli attori

Attore	Tecnici (Sviluppatori, Tester)	Amministratori (Team Leader, Manager)	Esterni (Stakeholder, Clienti)
Ruoli e Responsabilità	Segnalazione delle issues, assegnazione dei task, gestione delle priorità, sviluppo del codice, esecuzione e supervisione dei test, documentazione tecnica	Coordinamento del team, monitoraggio sull'avanzamento del progetto, reporting, supervisione generale del progetto	Monitoraggio di alto livello, verifica dei risultati, rilascio di feedback sui requisiti e sui prodotti consegnati
Competenze Tecniche	Conoscenza avanzata di linguaggi di programmazione, testing, version control, metodologie di sviluppo	Conoscenze tecniche di base/intermedie, competenze gestionali, pianificazione progetti, gestione delle risorse	Competenze di dominio e di business; non sono previste competenze tecniche o di programmazione specifiche

Attore	Tecnici (Sviluppatori, Tester)	Amministratori (Team Leader, Manager)	Esterni (Stakeholder, Clienti)
Frequenza e Modalità d'Uso	Utilizzo quotidiano, interazione con task e repository	Utilizzo regolare (giornaliero/settimanale), focus su dashboard, reportistica e gestione team	Utilizzo saltuario (settimanale/mensile), consultazione dello stato di avanzamento del progetto
Obiettivi	Completare e assegnare, collaborare con il team	Allocare risorse efficacemente, valutare le performance individuali degli utenti	Assicurarsi che il prodotto finale soddisfi i requisiti di business, valutare i progressi e avere visibilità sul progetto
Privilegi di Accesso	Creazione, lettura/-scrittura su task, accesso ai repository del progetto	Accesso completo al progetto, creazione, eliminazione task, gestione utenti, configurazione progetto, accesso a statistiche e reportistica avanzata	Accesso in sola lettura alle principali sezioni dell'applicazione

2.2 Requisiti Non Funzionali

Il sistema è composto da due moduli principali completamente disaccoppiati: un **frontend** e un **backend**.

2.2.1 Requisiti del Prodotto

Usabilità

- Il sistema deve offrire un'interfaccia utente semplice, intuitiva e moderna.
- Il frontend deve essere completamente **responsive**, garantendo un'esperienza d'uso ottimale sia su dispositivi desktop che su dispositivi mobili.
- La registrazione dei nuovi utenti deve avvenire mediante un sistema sicuro ad inviti tramite **token univoci** generati dagli amministratori, completabile in meno di 2 minuti.

- L'interfaccia grafica e i componenti visivi devono mantenere coerenza cromatica, tipografica e comportamentale in tutte le viste dell'applicazione.
- I nuovi utenti devono poter completare in autonomia almeno il 90% dei task operativi primari (creazione, aggiornamento, ricerca, filtraggio e ordinamento delle *issues*) entro 30 minuti dal primo accesso.
- Il sistema deve fornire messaggi di errore chiari e contestuali in risposta ad azioni non valide o fallimenti di validazione dei form.
- Le funzionalità principali e la navigazione devono essere facilmente individuabili senza necessità di consultare manuali d'uso.
- I titoli delle *issues* devono rispettare limiti di lunghezza prefissati per garantire una visualizzazione leggibile e coerente nelle dashboard e nelle tabelle.
- Ogni funzionalità fondamentale deve essere raggiungibile con un massimo di 3-5 click dalla homepage/dashboard principale.

Efficienza

- Il sistema deve garantire prestazioni elevate e tempi di caricamento contenuti sia da rete fissa che da rete mobile.
- Deve essere consentito il caricamento di allegati visivi e immagini a supporto delle segnalazioni con dimensione fino a **5 MB**, gestiti tramite servizi di cloud computing.

Affidabilità

- Il sistema deve garantire un **uptime del 99.5%** su base mensile durante gli orari lavorativi.
- Il backend deve gestire l'elaborazione delle eccezioni in modo centralizzato (*Global Exception Handler*) per prevenire crash inaspettati dell'applicazione e restituire risposte di errore strutturate.

Sicurezza

- Le password degli utenti non devono mai essere salvate in chiaro, ma sottoposte ad algoritmi di hashing sicuri (**BCrypt**).
- L'autenticazione e l'autorizzazione devono essere gestite tramite una sessione *stateless* basata su **JSON Web Tokens (JWT)**, validati ad ogni richiesta protetta.

- Il sistema deve garantire protezione integrata dalle principali vulnerabilità web:
 - **SQL Injection**: prevenuta tramite l'uso esclusivo di query parametrizzate e ORM
 - **XSS (Cross-Site Scripting)**
 - **CSRF (Cross-Site Request Forgery)**

Manutenibilità

- L'architettura del sistema deve essere fortemente modulare e disaccoppiata per facilitare modifiche, estensioni e manutenibilità.
- Il codice sorgente, ove necessario, deve essere documentato secondo gli standard **JavaDoc** per il backend e **TSDoc** per il frontend.
- Le API REST esposte dal backend devono essere documentate in modo interattivo e conforme allo standard **OpenAPI 3.0** tramite **Swagger UI**.

2.2.2 Requisiti Organizzativi

Ambientali e Infrastrutturali

- Le varie componenti del sistema devono essere completamente isolate e orchestrate tramite **container**, per garantire la riproducibilità immediata dell'ambiente di esecuzione.
- Il sistema deve essere predisposto per l'hosting su piattaforme di **cloud computing**, avvalendosi di servizi cloud dedicati per la gestione scalabile degli allegati.

Operativi e di Compatibilità

- L'interfaccia utente web deve garantire piena compatibilità e corretta resa grafica sulle versioni stabili dei principali browser desktop e mobile.
- Il sistema deve essere accessibile da qualsiasi dispositivo (desktop, tablet, smartphone) mediante un layout **responsive** e ottimizzato.
- Le API RESTful esposte dal backend devono seguire le specifiche HTTP ed essere testabili/interoperabili tramite client API standard (e.g. Bruno, Postman).

Di Sviluppo e Qualità del Codice

- L'intera codebase di frontend e backend deve essere versionata e tracciata in modo distribuito mediante **Git** su repository remoto (e.g. **GitHub**).
- Il flusso di sviluppo deve seguire le best practice di *feature-branching*: ogni modifica sostanziale deve essere integrata nel ramo principale tramite **Pull Request** soggetta ad approvazione previa *Code Review*.
- La qualità del codice, la presenza di vulnerabilità, i *code smells* e il rispetto degli standard architetturali devono essere analizzati automaticamente tramite **SonarQube** / **SonarCloud**.
- Lo stile e la formattazione del codice devono essere garantiti da linter e formatter automatici integrati nel workflow di build.
- La copertura dei test deve essere misurata in modo automatico durante la fase di build.

2.3 Modellazione dei Casi d'uso

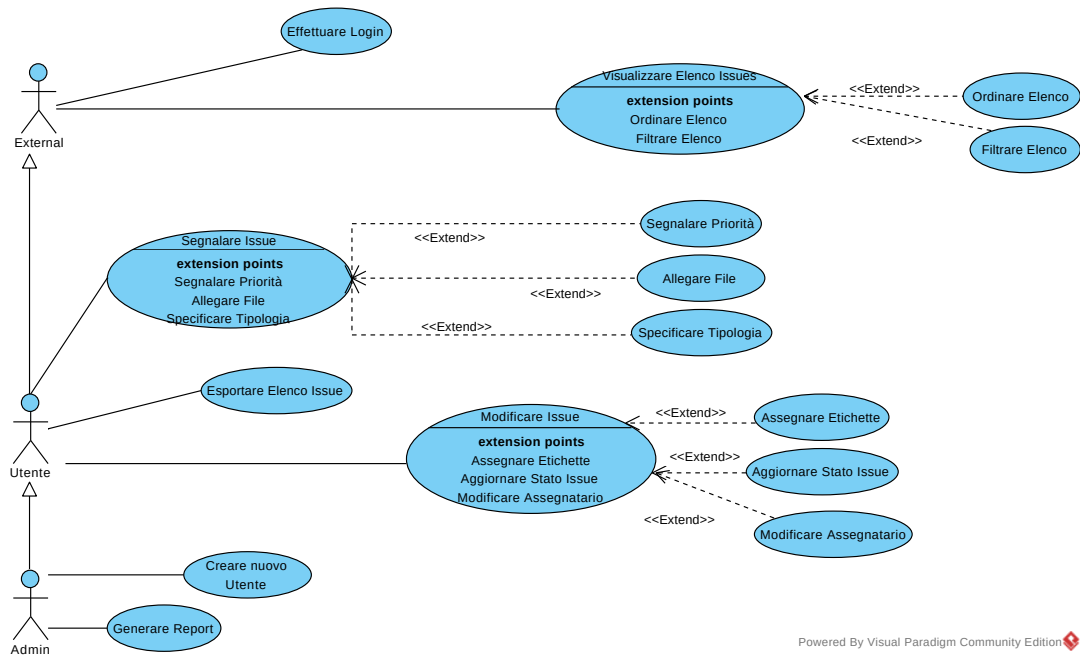


Figura 2.1: Diagramma dei casi d'uso principali di BugBoard26.

2.4 Casi d'uso Significativi

I casi d'uso presentati in questa sezione rappresentano alcune delle interazioni principali che caratterizzano il funzionamento del sistema BugBoard26.

La metodologia di documentazione adottata segue le linee guida contenute in *A Practical Style Guide and Templates Repository for Writing Effective Use Cases* [2], che fornisce, citando gli autori, *An opinionated style guide for use case writing* [3] attraverso il formalismo di Alistair Cockburn [1]. Questo approccio ci consente di descrivere in modo sistematico e coerente:

- L'obiettivo primario di ciascun caso d'uso dal punto di vista dell'attore
- Il flusso principale degli eventi e le relative condizioni di attivazione
- Gli eventuali flussi alternativi
- Le precondizioni necessarie e le postcondizioni attese

Per ogni caso d'uso significativo, forniamo una descrizione strutturata secondo il modello di Cockburn e una tabella delle proprietà rilevanti. Questo livello di dettaglio è necessario per garantire che le funzionalità chiave del sistema siano comprese in modo inequivocabile da tutte le parti interessate, dagli stakeholders agli sviluppatori.

Di seguito riportiamo i casi d'uso selezionati, che costituiscono alcune delle funzionalità *core* di BugBoard26 e rappresentano i principali flussi di interazione con il sistema.

2.4.1 UC1 - Creazione di una issue

ID - Nome dello UC	1 - Creazione di una issue [4]
Data di creazione	11/10/2025
Attore primario	Utente
Trigger	L'utente indica di voler creare una nuova issue
Descrizione	L'utente ha trovato un problema e intende aprire una nuova issue per tenere traccia di esso.

Precondizioni	PRE-1. L'utente è stato registrato da un amministratore ed ha effettuato correttamente l'autenticazione PRE-2. Il sistema ha caricato la pagina di riepilogo delle issues
Postcondizioni	POST-1. La nuova issue è salvata nel sistema POST-2. La nuova issue è visibile nella vista di riepilogo delle issue
Scenario di successo	1. L'utente indica di voler segnalare una nuova issue (M1) 2. Il sistema apre la modale di creazione di una nuova issue (M2) 3. L'utente inserisce il titolo della issue (M3) 4. L'utente indica i dettagli della issue nella descrizione (M4) 4a. [Opzionale] L'utente sceglie una priorità per la issue 4b. [Opzionale] L'utente può allegare un'immagine 4c. [Opzionale] L'utente sceglie un tipo per la issue 5. L'utente segnala di aver terminato l'inserimento dei dati richiesti (M5) 6. Il sistema valida i dati inseriti dall'utente 7. Il sistema salva la nuova issue 8. Il sistema mostra un messaggio di successo (M6)

Continua nella pagina successiva

Estensioni	4a. Scelta della priorità per la issue: 4a1. L'utente clicca sul menù "priorità" (M9) 4a2. Il sistema apre un menù con le possibili priorità (M10) 4a3. L'utente sceglie una tra le alternative (M11) 4a4. Il sistema mostra l'alternativa scelta (M12) 4a5. Il flusso ritorna al termine del punto 4 dello scenario di successo 4b. Caricamento di un'immagine: 4b1. L'utente clicca sul bottone per caricare un'immagine (M13) 4b2. Il sistema apre il file explorer (M14) 4b3. L'utente seleziona un'immagine dal proprio dispositivo (M15, M16) 4b4. Il sistema valida il formato e la dimensione dell'immagine 4b5. Il sistema sostituisce l'icona di "allega" con un messaggio di successo (✓) e aggiunge un bottone di eliminazione accanto alla label "immagine"(M17) 4b7. Il flusso torna al termine del punto 4a dello scenario di successo 4b3a. L'utente annulla il caricamento dell'immagine: 4b3a1. L'utente clicca sul bottone "annulla" 4b3a2. Il flusso ritorna al termine del punto 4 dello scenario di successo 4b4a. Errore di validazione dell'immagine: 4b4a1. Il sistema mostra un messaggio di errore e invita l'utente a riprovare (M18) 4b4a2. Il flusso ritorna al termine del punto 4 dello scenario di successo 4b6a. Eliminazione dell'immagine allegata: 4b6a1. L'utente clicca sul bottone di eliminazione 4b6a2. Il sistema elimina l'immagine allegata 4b6a3. Il sistema sostituisce l'icona di successo (✓) con il bottone "allega" e rimuove il bottone di eliminazione 4b6a4. Il flusso ritorna al termine del punto 4 dello scenario di successo
-------------------	---

Continua nella pagina successiva

Estensioni	4c. Scelta del tipo di issue: 4c1. L'utente clicca sul menù "tipo" (M19) 4c2. Il sistema apre il menù con i vari tipi di issues (M20) 4c3. L'utente sceglie un tipo di issue (M21) 4c4. Il sistema mostra il tipo scelto (M22) 4c5. Il flusso ritorna al termine del punto 4 dello scenario di successo 6a. Errore nella validazione dell'input (titolo): 6a1. L'utente inserisce un titolo non valido 6a2. Il sistema evidenzia il campo del titolo e mostra un messaggio all'utente, segnalando la non validità del titolo scelto (M7) 6a3. L'utente modifica il titolo, inserendone uno corretto (M3) 6a4. Il flusso ritorna al punto 4 dello scenario di successo 6b. Errore nella validazione dell'input (descrizione): 6b1. L'utente inserisce una descrizione non valida 6b2. Il sistema evidenzia il campo della descrizione e mostra un messaggio all'utente, segnalando la non validità della descrizione scelta (M8) 6b3. L'utente inserisce una descrizione valida (M4) 6b4. Il flusso ritorna al punto 5 nello scenario di successo 7a. Errore nel salvataggio della issue: 7a1. Il sistema riscontra un errore durante il salvataggio della issue 7a2. Il sistema mostra uno specifico messaggio di errore, invitando l'utente a riprovare (M23) 7a3. Il flusso ritorna al termine del punto 4 nello scenario di successo
Priorità	Alta
Frequenza d'uso	Presumibilmente tutti gli utenti, tra 1 e 5 volte al giorno per utente
Informazioni associate	Mostrato nella tabella 2.4

Nome della proprietà	Tipo	Regole di validazione
Titolo	String	Obbligatorio, compreso tra 1 e 256 caratteri
Descrizione	String	Obbligatorio, compreso tra 1 e 1000 caratteri
Priorità	String	Facoltativo, valori possibili: - Lowest - Low - Medium - High - Highest
Immagine	File (image)	Facoltativo, dimensione massima: 5 MB
Tipo	String	Facoltativo, valori possibili: - Question - Bug - Documentation - Feature - Other

Tabella 2.4: Regole di validazione delle proprietà

2.4.2 UC2 - Associazione di etichette personalizzate

Descrizione Strutturata (Formalismo di A. Cockburn)

Tabella 2.5: UC-2 Associazione Etichette – Template di Cockburn

Campo	Valore
ID - Nome dello UC	2 - Associazione etichette per i Bug [5]
Data di creazione	14/10/2025
Attore primario	Utente
Trigger	L'Utente modifica un bug e indica di voler associare una o più etichette
Descrizione	L'Utente desidera associare un numero variabile di etichette ad un bug per facilitarne la classificazione.
Precondizioni	• PRE-1. L'Utente è registrato nel sistema. • PRE-2. Il bug selezionato esiste nel sistema.
Postcondizioni	• POST-1. Una o più etichette saranno assegnate al bug scelto dall'utente.
Scenario di successo	1. L'Utente seleziona un bug esistente 2. Il Sistema visualizza i dettagli del bug, comprese eventuali etichette già associate. 3. L'Utente indica di voler aggiungere una o più etichette al bug. 4. Il Sistema salva le etichette associate e aggiorna la visualizzazione del bug 5. Il sistema notifica l'Utente che le etichette sono state aggiunte con successo
Estensioni	9a. Errore nel salvataggio delle nuove etichette 9a1. Il sistema riscontra un errore durante il salvataggio delle etichette associate. 9a2. Il sistema mostra uno specifico messaggio di errore. 9a3. Il flusso ritorna al punto 1 nello scenario di successo.
Priorità	Alta
Frequenza d'uso	Media

2.4.3 UC3 – Ordinamento delle Issue

Descrizione Strutturata (Formalismo di A. Cockburn)

Tabella 2.6: UC-3 Ordinamenti – Template di Cockburn

Campo	Valore
ID - Nome dello UC	3 - Ordinamento [6]
Data di creazione	14/10/2025
Attore primario	Utente
Trigger	L'utente indica di voler ordinare l'elenco delle issues.
Descrizione	L'utente vuole ordinare le issues presenti nell'elenco tramite un criterio specifico.
Precondizioni	• PRE-1. L'utente è stato registrato da un amministratore ed ha effettuato correttamente l'autenticazione. • PRE-2. Il sistema ha caricato la pagina di riepilogo delle issues.
Postcondizioni	L'elenco delle issues è riordinato.
Scenario di successo	1. L'utente visualizza la tabella contenente le issues con le relative colonne. 2. L'utente clicca sull'intestazione di una colonna (es. stato, priorità, data creazione) per applicare un ordinamento. 3. Il sistema riordina la tabella in base ai valori della colonna selezionata e mostra un indicatore visivo della direzione dell'ordinamento (es. ↑ o ↓). 4. <i>[Opzionale]</i> Se l'utente clicca nuovamente su una colonna già ordinata, il sistema inverte la direzione dell'ordinamento o ripristina l'ordine predefinito. 5. Il sistema mostra la tabella aggiornata e, se previsto, memorizza i criteri di ordinamento nella sessione o nell'URL.

Continua nella pagina successiva

Campo	Valore
Estensioni	2b. Ordinamento su colonna non ordinabile 2b1. L'utente clicca su una colonna che non supporta l'ordinamento (es. colonna tag). 2b2. Il sistema ignora il click senza modificare la tabella. 2b3. Il flusso ritorna al punto 2 dello scenario di successo. 3a. Ordinamento crescente/decrescente 3a1. L'utente clicca una seconda volta sulla stessa intestazione di colonna. 3a2. Il sistema inverte la direzione dell'ordinamento (da crescente a decrescente o viceversa). 3a3. Il sistema aggiorna l'indicatore visivo (freccia ↑ / ↓) e ricarica la tabella. 3a4. Il flusso ritorna al termine del punto 3 dello scenario di successo. 4b. Rimozione di un ordinamento attivo 4b1. L'utente clicca nuovamente su una colonna ordinata fino a tornare alla condizione "nessun ordine". 4b2. Il sistema rimuove l'indicatore visivo e ripristina l'ordinamento predefinito. 4b3. Il flusso ritorna al termine del punto 3 dello scenario di successo.
Priorità	Media
Frequenza d'uso	Tutti gli utenti, tra 1 e 3 volte al giorno per utente
Informazioni associate	Mostrato nella Tabella 2.7

Tabella 2.7: UC-3 Ordinamenti – Proprietà e Regole di Validazione

Nome della Proprietà	Tipo	Regole di validazione	Note
Colonne ordinabili	String	Obbligatorio, click sulla colonna per ordinarlo	Definisce su quali campi è permesso applicare l'ordinamento (nome, <i>stato</i> , <i>priorità</i> , <i>data creazione</i>)

Continua nella pagina successiva

Nome della Proprietà	Tipo	Regole di validazione	Note
Direzione ordinamento	Boolean (↑/↓)	Invertibile dall'utente	Indica se l'ordinamento è crescente o decrescente
Indicatore visivo	Icona UI	Aggiornato automaticamente	Mostra la direzione e il livello dell'ordinamento accanto all'intestazione
Colonne non ordinabili	String	Click ignorato	Colonna Tag

3 Definizione dell'architettura software

3.1 Introduzione

In questa sezione procederemo a definire l'architettura software della piattaforma.

Data la natura dell'applicazione, i casi d'uso individuati e le funzionalità che il sistema dovrà offrire, è stato deciso di sviluppare una web app, accessibile sia da desktop che da mobile, in maniera da permettere all'intero team di sviluppo di :

- segnalare comodamente i bug da un'interfaccia desktop, inserendo delle descrizioni approfondite
- monitorare lo stato delle diverse segnalazioni, sia da desktop che da mobile, avendo a disposizione molte segnalazioni nella stessa schermata nella versione desktop, ma anche visualizzarne una nel dettaglio tramite l'interfaccia mobile
- garantire la sincronizzazione e la centralizzazione dei dati, indipendentemente dal dispositivo utilizzato per l'accesso

3.2 Architettura Logica

In conformità con i requisiti non funzionali e i vincoli di progetto, l'architettura di *Bug-Board26* è di tipo distribuito, basata sul paradigma Client-Server e sullo stile architetturale *REpresentational State Transfer (REST)*.

Il sistema è decomposto in due macro-componenti logicamente e fisicamente separati, che comunicano esclusivamente tramite protocollo HTTP scambiando messaggi in formato JSON:

- **Frontend (Client):** Responsabile della logica di presentazione e dell'interazione con l'utente. Implementato come *Single Page Application (SPA)*, questo modulo è completamente disaccoppiato dal backend, garantendo la possibilità di evoluzione indipendente o la sostituzione dell'interfaccia senza impatti sulla logica di business.
- **Backend (Server):** Responsabile della logica di business, della sicurezza e della persistenza dei dati. Il backend espone interfacce di programmazione (API) pubbliche e private, operando in modalità *stateless*: ogni richiesta contiene tutte le informazioni necessarie per essere elaborata, garantendo la scalabilità orizzontale.

3.3 Pattern Architeturale Backend

Il modulo Backend è strutturato internamente secondo il pattern **Layered Architecture** per garantire la *Separation of Concerns*. Il flusso di elaborazione di una richiesta attraversa i seguenti livelli:

- **Interface Layer (Controller):** Punto di ingresso delle richieste REST. Gestisce la validazione sintattica degli input (DTO) e la mappatura delle risposte HTTP.
- **Business Logic Layer (Service):** Implementa le regole di dominio, i controlli di sicurezza applicativa e il coordinamento delle transazioni. È isolato dai dettagli dell'interfaccia utente e della persistenza.
- **Data Access Layer (Repository):** Astrae l'accesso al database relazionale tramite un'interfaccia. Fornisce i metodi per la persistenza e il recupero delle entità, disaccoppiando la logica di business dal linguaggio SQL specifico.

3.4 Stack tecnologico backend

Componente	Tecnologia Utilizzata	Motivazione
Linguaggio Backend	Java	Linguaggio fortemente tipizzato, Object-Oriented, utilizzato dal 99% delle aziende [7]
Framework Backend	Spring Boot 4	Offre Dependency Injection, semplifica la scrittura del codice rispetto a Java Spring
Database	PostgreSQL	DBMS relazionale robusto, affidabile, sicuro, open source e veloce
Object-Relational Mapping (ORM)	Spring Data JPA	Astrazione necessaria per la gestione della persistenza orientata agli oggetti, riducendo il codice boilerplate e prevenendo nativamente SQL Injection
Container	Docker & Podman	Ambiente indipendente che garantisce un funzionamento coerente su ogni ambiente

3.5 Pattern Architetture Frontend

Il modulo Frontend è progettato come una *Single Page Application* (SPA) basata su una **Component-Based Architecture** e sull'architettura *Standalone* di Angular. L'interfaccia utente è decomposta in unità funzionali indipendenti, fortemente coese e riutilizzabili (Componenti), organizzate in una struttura gerarchica.

Per garantire la manutenibilità, l'estensibilità e il disaccoppiamento del codice, viene applicata la netta separazione tra:

- **Container Components (Smart):** Responsabili del recupero dei dati tramite servizi, dell'interazione con le logiche di business applicative e della gestione dello stato complesso della vista.
- **Presentational Components (Dumb):** Responsabili esclusivamente del rendering grafico e dell'intercettazione degli eventi utente sul DOM. Essi sono privi di dipendenze esterne o accoppiamenti con la logica di dominio. Un esempio significativo all'interno del sistema è il componente `PaginationComponent`, responsabile esclusivamente del rendering dei controlli di paginazione e dell'emissione degli eventi di cambio pagina verso il componente padre

Gestione del Flusso Dati e Reattività Tramite Signals Il passaggio dei dati e delle configurazioni tra componenti sfrutta gli **Angular Signals**. In particolare, la ricezione dei parametri di presentazione avviene tramite *Signal Inputs* (`input()`). Questo paradigma permette al framework di tracciare in maniera granulare le dipendenze dei dati, ottimizzando il ciclo di *Change Detection* ed eseguendo l'aggiornamento del DOM senza la necessità di cicli di controllo globali. Al contempo, la gestione delle operazioni asincrone e della comunicazione con le API REST del Backend è governata secondo l'**Observer Pattern** tramite la libreria *RxJS*.

3.6 Dettaglio Tecnologie Frontend

A integrazione delle scelte generali di progetto, si motivano le tecnologie specifiche adottate per lo sviluppo del livello di presentazione:

- **Framework: Angular (v21).** L'adozione dell'ultima release stabile è motivata dall'integrazione nativa dell'architettura standalone e dall'utilizzo dei Signals per la gestione efficiente e reattiva dello stato. L'intero ecosistema si basa sulla natura *strongly-typed* di TypeScript, garantendo perfetta coerenza strutturale con i modelli dati (DTO) esposti dal backend Java e minimizzando gli errori a runtime.

- **Styling: Bootstrap (v5.3+), Bootstrap Icons & SCSS (Dart Sass).** In opposizione alla scrittura di codice CSS atomico o ridondante nei singoli componenti, si è optato per la centralizzazione del design system. Sfruttando la moderna sintassi a moduli di Dart Sass (@use), le variabili strutturali del framework (colore primario \$primary impostato a ■ #005BB0, spaziature, ombreggiature e raggio dei bordi) vengono configurate e iniettate a monte del processo di compilazione tramite il costrutto with. In questo modo, i componenti nativi di Bootstrap assorbono le direttive senza generare debito tecnico o fogli di stile duplicati, rispettando pienamente il principio *Don't Repeat Yourself* (DRY).

3.7 Architettura Fisica e Deployment

L'architettura fisica di BugBoard26 segue un modello client-server distribuito. L'infrastruttura è ospitata sulla piattaforma **Microsoft Azure** e sfrutta la containerizzazione tramite Docker per garantire portabilità, isolamento e semplicità di deployment. I diversi componenti comunicano esclusivamente tramite protocollo *HTTPS* e *API REST*.

Fisicamente, il deployment del sistema si articola nei seguenti nodi e ambienti di esecuzione:

Nodo Cloud – Application Server (Backend): Il backend è distribuito all'interno di un container Docker ospitato sull'infrastruttura Microsoft Azure. Il container esegue l'applicazione sviluppata in *Java 21* mediante il framework *Spring Boot 4*, esponendo un insieme di API REST utilizzate dal frontend. Il modulo applicativo gestisce la logica di business, l'autenticazione e autorizzazione degli utenti tramite cookie JWT, la persistenza dei dati e l'interazione con i diversi servizi cloud.

Nodo Cloud – Database Server: La persistenza dei dati è affidata ad un'istanza di *Azure Database for PostgreSQL Flexible Server*, che ospita il database relazionale dell'applicazione. L'accesso ai dati avviene tramite *Spring Data JPA* e l'ORM *Hibernate*, consentendo una gestione trasparente delle entità persistenti e delle relative transazioni.

Nodo Cloud – Object Storage: Gli allegati associati alle issue (ad esempio immagini e screenshot) sono memorizzati mediante il servizio *Azure Blob Storage*. Il backend gestisce le operazioni di upload, download ed eliminazione dei file attraverso le API messe a disposizione dal servizio, mentre nel database vengono conservati esclusivamente i riferimenti ai file memorizzati.

Nodo Cloud – Frontend Server: La Single Page Application è distribuita come applicazione statica sull'infrastruttura Azure. Tale nodo ha il solo compito di servire le risorse statiche (HTML, CSS, JavaScript e immagini) ai browser degli utenti, demandando tutta la logica applicativa al backend mediante API REST.

Nodi Client (Dispositivi Utente): L'accesso al sistema avviene tramite browser web eseguiti su dispositivi desktop o mobili. All'avvio dell'applicazione viene scaricata ed eseguita la Single Page Application sviluppata in *Angular 21*, realizzata in *TypeScript* e stilizzata mediante *SCSS* e *Bootstrap 5*. Il frontend comunica esclusivamente con il backend mediante richieste HTTPS verso le API REST, consentendo un completo disaccoppiamento tra i due moduli.

3.8 Descrizione dello schema per la persistenza dati

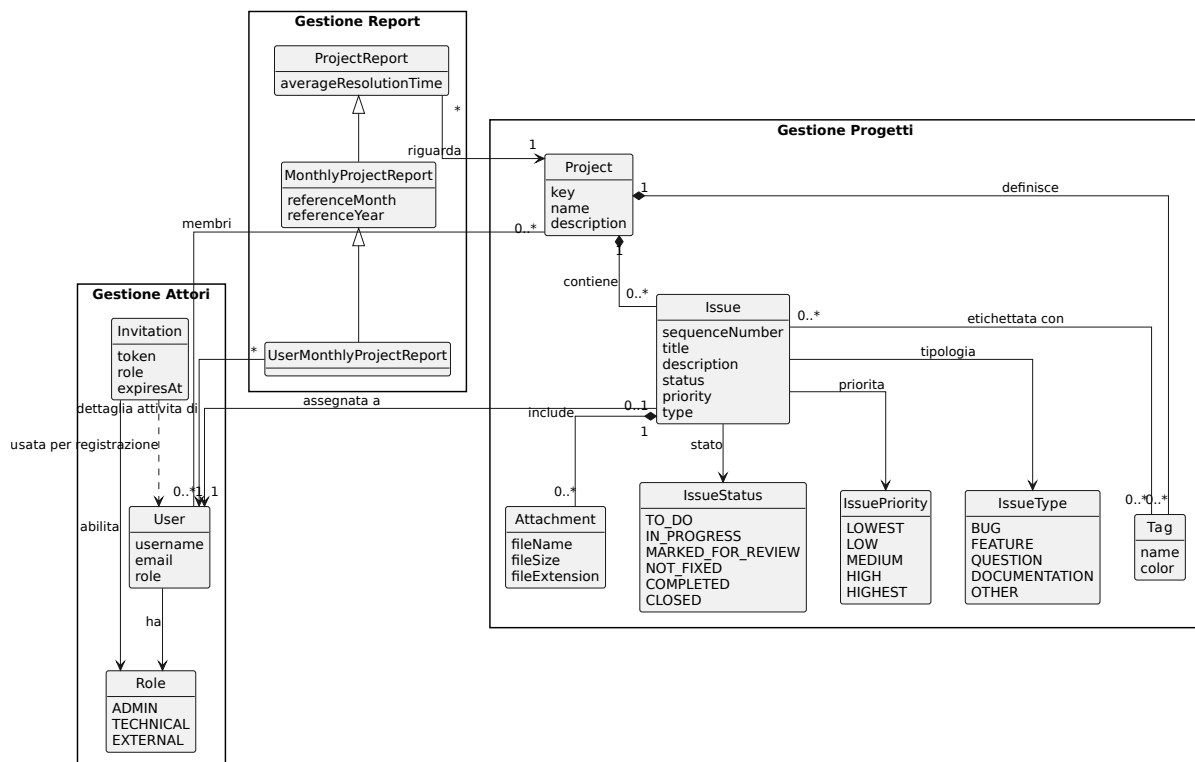


Figura 3.1: Schema di persistenza dei dati

4 Definizione dei Design Patterns

L'architettura del sistema si avvale dell'utilizzo di design pattern consolidati per garantire la manutenibilità, l'estensibilità e la robustezza del codice. L'adozione di tali soluzioni permette di separare le responsabilità dei componenti, facilitando l'evoluzione del software in risposta a nuovi requisiti senza compromettere le funzionalità esistenti. Di seguito vengono analizzati i pattern principali implementati nel progetto.

4.1 Singleton Pattern

Per l'accesso alla base dati e la gestione delle connessioni a PostgreSQL, l'architettura sfrutta implicitamente il Singleton Pattern per garantire un accesso centralizzato e ottimizzato. Tuttavia, in aderenza alle best practice del framework Spring Boot, si è evitato di implementare manualmente il classico pattern GoF (con costruttore privato e metodo statico). Si è scelto invece di demandare il ciclo di vita dei componenti al framework, sfruttando la Dependency Injection. I Repository di Spring Data JPA (es. `IssueRepository`, `UserRepository`) sono generati e gestiti dal container Inversion of Control di Spring con scope Singleton di default. Questo approccio unisce i vantaggi prestazionali del Singleton (un solo gestore del Connection Pool in memoria) a quelli del basso accoppiamento garantito dalla Dependency Injection, rendendo inoltre le classi facilmente testabili in isolamento.

4.2 Dependency Injection e Inversion of Control (IoC)

La Dependency Injection rappresenta il cardine architettonico dell'intero sistema. Invece di permettere ai componenti di istanziare autonomamente le proprie dipendenze (creando un forte accoppiamento), il controllo viene delegato al container di Spring Boot. L'implementazione segue il principio dell'*Injection tramite costruttore*:

- Il `IssueController` riceve un'istanza di `IssueService`.
- Il `IssueService` riceve i repository necessari (`IssueRepository`, `TagRepository`).

Questo approccio garantisce un **basso accoppiamento** e un'alta testabilità: è infatti possibile iniettare *Mock Objects* durante i test di unità per isolare la logica di business senza interagire con il database reale. Inoltre, la DI facilita l'implementazione dello *Strategy Pattern* precedentemente descritto, permettendo al framework di selezionare dinamicamente l'implementazione corretta della strategia di esportazione al momento dell'esecuzione.

4.3 Strategy Pattern

Il *Strategy Pattern* è stato adottato per la gestione dell'esportazione delle *Issue*. Attraverso l'interfaccia `IssueExportStrategy`, il sistema definisce una famiglia di algoritmi di esportazione intercambiabili, ciascuno responsabile della generazione dei dati in uno specifico formato. Il servizio `IssueService` svolge il ruolo di *Context*, mantenendo un riferimento alla strategia selezionata senza dipendere dalla sua implementazione concreta.

Attualmente il sistema implementa un'unica strategia concreta, dedicata all'esportazione in formato CSV. L'adozione dello *Strategy Pattern*, nonostante la presenza di un solo formato disponibile, rappresenta una scelta progettuale orientata all'estensibilità: in futuri aggiornamenti sarà infatti possibile introdurre nuovi formati di esportazione (ad esempio PDF o JSON) semplicemente implementando nuove classi che realizzino l'interfaccia `IssueExportStrategy`, senza apportare modifiche al codice del servizio.

Questa soluzione rispetta il principio *Open/Closed* (OCP) dei principi SOLID, secondo cui il software deve essere aperto all'estensione ma chiuso alla modifica, favorendo la manutenibilità e riducendo l'accoppiamento tra la logica applicativa e le modalità di esportazione.

5 Processo di Sviluppo e CI/CD

5.1 Gestione del Versioning

Lo sviluppo del progetto **BugBoard26** è stato gestito tramite *Git*, utilizzando *GitHub* come piattaforma di hosting del repository e collaborazione tra i membri del team.

Per organizzare lo sviluppo parallelo delle funzionalità è stato adottato il *Feature Branch Workflow*, una strategia di branching che prevede la creazione di un branch dedicato per ogni nuova funzionalità o modifica significativa.

In particolare, il repository è organizzato secondo la seguente struttura:

- **main**: contiene esclusivamente versioni stabili e pronte per il rilascio;
- **development**: rappresenta il ramo di integrazione delle nuove funzionalità completate;
- **feature/***: branch temporanei utilizzati per lo sviluppo di singole funzionalità.
- **bug/***: branch temporanei utilizzati per la risoluzione di problemi individuati nel sistema. Consentono di isolare le modifiche dal normale sviluppo delle funzionalità.

Ogni nuova attività viene sviluppata all'interno di un branch dedicato, creato a partire dal ramo di sviluppo. Una volta completata la modifica, il codice viene integrato attraverso una *Pull Request* verso il branch di destinazione.

Prima del merge definitivo, ogni Pull Request è stata sottoposta a revisione da parte di almeno un altro componente del team. Tale processo ha permesso di verificare la correttezza delle modifiche introdotte, migliorare la qualità del codice e ridurre il rischio di introdurre regressioni nel progetto.

5.2 Continuous Integration

Per automatizzare le attività di verifica del codice è stato adottato un approccio di *Continuous Integration* mediante *GitHub Actions*.

Le pipeline configurate vengono eseguite automaticamente in seguito a modifiche sui branch principali del progetto: **main**, **development**, **feature/***.

Inoltre, l'esecuzione viene attivata anche in occasione dell'apertura, aggiornamento o riapertura di una Pull Request.

L'obiettivo principale della Continuous Integration è garantire che ogni modifica introdotta nel repository sia verificata automaticamente prima di essere integrata nel codice principale.

5.3 Workflow GitHub Actions Implementati

Sono stati definiti diversi workflow automatici, ciascuno responsabile di una specifica fase del processo di sviluppo.

5.3.1 Verifica della formattazione del codice

Il workflow *Verify Code Formatting with Prettier* esegue automaticamente un controllo sulla formattazione del codice frontend mediante il tool *Prettier*.

Questa verifica permette di mantenere uno stile uniforme all'interno del progetto e di evitare differenze di formattazione tra i diversi contributori.

Sono stati inoltre definiti specifici script `npm` per semplificare l'utilizzo di *Prettier* durante lo sviluppo locale. In particolare, sono stati introdotti i seguenti comandi:

- `format:check`: esegue una verifica della formattazione dei file senza apportare modifiche, segnalando eventuali difformità rispetto alle convenzioni definite dal progetto.
- `format:fix`: applica automaticamente le correzioni di formattazione ai file del progetto.

La presenza di questi script consente agli sviluppatori di verificare e correggere autonomamente lo stile del codice prima dell'apertura di una Pull Request, riducendo il numero di modifiche esclusivamente stilistiche durante la fase di revisione.

5.3.2 Build automatica

Il workflow di *Build* verifica automaticamente la corretta compilazione dell'applicazione.

Durante questa fase vengono controllati eventuali errori di compilazione del codice sorgente, garantendo che le modifiche introdotte non compromettano la generazione degli artefatti software.

Per il backend, la compilazione viene eseguita tramite *Maven* utilizzando l'ambiente *Java 21*.

5.3.3 Analisi statica del codice tramite SonarQube

È stato implementato un workflow dedicato all'integrazione con *SonarQube Cloud* per l'analisi automatica della qualità del codice backend.

La pipeline esegue:

- il checkout del repository;
- la configurazione dell'ambiente Java 21;
- il ripristino delle dipendenze Maven tramite cache;
- la compilazione e l'esecuzione dei test automatici;
- l'analisi statica del codice tramite il plugin Maven di SonarQube.

L'analisi consente di individuare automaticamente:

- potenziali bug;
- vulnerabilità di sicurezza;
- duplicazioni di codice;
- *code smell*;
- metriche relative alla manutenibilità del software.

Il controllo viene eseguito sia sui branch di sviluppo sia sulle Pull Request, consentendo di individuare eventuali problemi prima dell'integrazione definitiva del codice.

Di seguito il report aggiornato all'ultima versione (Figura 5.1).

5.4 Continuous Deployment

Il processo di rilascio dell'applicazione è stato automatizzato mediante una pipeline di deployment verso l'infrastruttura cloud Azure.

Al completamento positivo delle verifiche automatiche, il sistema procede alla generazione degli artefatti necessari e al loro rilascio nell'ambiente di produzione.

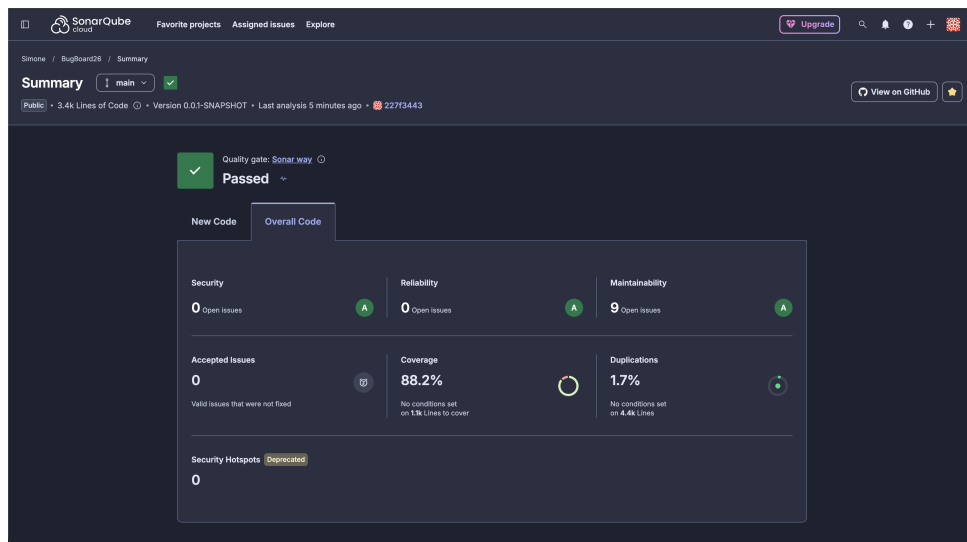


Figura 5.1: Report dell'analisi di SonarQube aggiornato all'ultima versione.

In particolare, il backend viene distribuito tramite container Docker, garantendo un ambiente di esecuzione isolato e riproducibile.

L'automazione del deployment permette di ridurre gli interventi manuali, diminuendo il rischio di errori durante il rilascio e garantendo maggiore affidabilità nel processo di aggiornamento del sistema.

5.5 Gestione della Concorrenza delle Pipeline

Per evitare l'esecuzione contemporanea di pipeline ridondanti è stata configurata una gestione della concorrenza tramite GitHub Actions.

Nel caso in cui una nuova esecuzione venga avviata sullo stesso branch o sulla stessa Pull Request, eventuali workflow precedenti ancora in esecuzione vengono automaticamente annullati.

6 Attività di Testing

L'attività di verifica applicata al backend della piattaforma **BugBoard26** adotta una strategia rigorosa basata sul combinato disposto di tecniche di *Black Box Testing* (orientato alle specifiche funzionali e al partizionamento del dominio d'ingresso) e *White Box Testing* (orientato alla copertura strutturale dei rami decisionali).

I test automatici sono sviluppati in **Java 21** avvalendosi del framework **JUnit 5 (Jupiter)** per la struttura delle suite di prova e le asserzioni, di **Mockito 5** per l'isolamento dei componenti di servizio tramite la generazione dinamica di *Test Doubles* (Mock), e di **JaCoCo** per la misurazione oggettiva delle metriche di copertura del codice.

6.1 Strategia Generale di Testing

La strategia di verifica adottata per il backend si concentra principalmente sul livello di **Unit Testing del Service Layer**.

Tale approccio risponde al principio dell'isolamento dei componenti: ogni classe di servizio viene verificata atomicamente disaccoppiandola completamente dall'infrastruttura di persistenza (*Database PostgreSQL/H2*), dal livello web REST e dai servizi cloud esterni (*Azure Blob Storage*).

L'architettura delle verifiche automatizzate si articola nei seguenti aspetti fondamentali:

1. **Unit Testing del Service e Domain Layer:** Costituisce il cuore dell'attività di verifica. Sono test eseguiti ad altissima velocità che validano la logica applicativa, il rispetto delle regole di dominio (es. gestione degli stati delle Issue, assegnazione membri, calcolo dei parametri di default e validazione degli allegati). Tutte le dipendenze da componenti esterni (quali **Repository** JPA, mapper, etc.) sono simulate tramite le annotazioni `@Mock` e `@InjectMocks` di Mockito.
2. **Isolamento dell'Ambiente e Indipendenza dai Dati:** L'utilizzo dei mock garantisce la ripetibilità e il determinismo dei test. Non effettuando accessi reali a basi di dati o a risorse di rete fisiche, i test risultano immuni da effetti collaterali dovuti allo stato del database o a fallimenti dell'infrastruttura di rete.
3. **Analisi della Copertura del Codice (*Code Coverage*):** L'efficacia della suite di test viene quantificata a livello strutturale misurando la *Statement Coverage* (copertura delle istruzioni) e la *Branch Coverage* (copertura delle decisioni) sui

metodi di servizio, assicurando il superamento della soglia minima di qualità definita tramite SonarQube.

6.2 Tecniche di Progettazione dei Test

La progettazione dei casi di test in *BugBoard26* unisce l'approccio funzionale (*Black Box*) per la definizione degli scenari d'uso e dei vincoli di interfaccia, all'approccio strutturale (*White Box*) per il raggiungimento di un'elevata copertura delle istruzioni e dei rami condizionali.

6.2.1 Black Box Testing

Nel Black Box Testing il sistema viene analizzato considerando esclusivamente le sue specifiche d'interfaccia (ingressi ed uscite attese), ignorando la struttura interna del codice. Sono state applicate le seguenti tecniche:

Partizionamento in Classi di Equivalenza (Equivalence Partitioning)

Il dominio di ciascun parametro di ingresso è stato suddiviso in classi di equivalenza valide (che devono produrre un esito positivo) e non valide (che devono generare un rifiuto o un'eccezione controllata):

- **Identificativo Progetto (projectKey):**
 - *Classe Valida:* Chiave di un progetto esistente nel database (es. "FRONT").
 - *Classe Non Valida:* Chiave inesistente o nulla (es. "NOT_EXISTENT"), che solleva `ResourceNotFoundException`.
- **Campi Testuali (title, description):**
 - *Classe Valida:* Stringhe non vuote e conformi ai limiti di lunghezza.
 - *Classe Non Valida:* Stringhe vuote, contenenti soli spazi bianchi o `null`, che violano l'annotazione `@NotBlank` restituendo HTTP 400 Bad Request.
- **Assegnatario (assigneeUsername):**
 - *Classe Nessun Assegnatario:* Valore `null` o vuoto, che imposta l'assegnatario della issue a `null`.
 - *Classe Utente Interno Valido:* Username di un utente esistente con ruolo interno (TECHNICAL, ADMIN).

- *Classe Utente Esterno Non Consentito*: Username di un utente con ruolo `EXTERNAL`, che solleva `OperationNotAllowedException`.
- **Collezione Tag (tags)**:
 - *Classe Lista Valida*: Lista di oggetti `TagResponse` i cui ID corrispondono a tag associati al medesimo progetto.
 - *Classe Lista Vuota / Null*: Lista assente o vuota, che si traduce in un'associazione vuota senza errori.
 - *Classe Tag Cross-Project*: Tentativo di associare tag appartenenti ad altri progetti, che solleva `OperationNotAllowedException`.

Analisi dei Valori Limite (Boundary Value Analysis - BVA)

La BVA è stata impiegata per testare il comportamento del sistema in corrispondenza dei confini delle classi di equivalenza:

- **Lunghezza Stringhe**: Test su stringhe di titolo a lunghezza minima valida (1 carattere) e immediatamente al di fuori (0 caratteri).
- **Numerazione Sequenziale (sequenceNumber)**: Generazione del primo numero sequenziale per un progetto privo di issue (stato di confine con `maxSequenceNumber = null`, da cui scaturisce `sequenceNumber = 1`) e incremento progressivo per issue successive.
- **Paginazione delle Issue**: Selezione della prima pagina (`page = 0`), dimensione di pagina unitaria (`size = 1`) e richieste di pagine oltre il limite massimo di elementi totali.

6.2.2 White Box Testing

La tecnica White Box sfrutta la conoscenza della struttura interna del codice Java per definire casi di test in grado di attraversare specifici percorsi di esecuzione (*Control Flow Graph*).

- **Branch Coverage (Copertura dei Rami)**: Ogni istruzione condizionale (`if/else`, operatori ternari, istruzioni `switch`) è stata testata per tutti i possibili esiti booleani. Ad esempio, nel metodo `createIssue`, viene testato sia il ramo in cui `request.getStatus()` è valorizzato (preservando lo stato fornito), sia il ramo in cui è `null` (assegnando il valore di default `IssueStatus.TO_DO`).

- **Exception Path Testing (Copertura dei Percorsi di Eccezione):** I blocchi che sollevano eccezioni di dominio (`ResourceNotFoundException`, `OperationNotAllowedException`) sono stati verificati puntualmente tramite l'asserzione `assertThrows` di JUnit 5, garantendo che le eccezioni vengano lanciate con il messaggio d'errore corretto e prima che vengano effettuate mutazioni sullo stato del database.

6.3 Architettura dei Test Backend

6.3.1 Unit Testing del Service Layer

I test di unità del service layer sono annotati con `@ExtendWith(MockitoExtension.class)`. Questo approccio isola completamente il service under test iniettando mock per i JPA repository.

Analisi dei Metodi

Sono stati individuati tre metodi non banali del Service Layer, aventi ciascuno almeno due parametri e logica di business complessa, coperti da test unitari automatici. Di seguito è riportata la suite di test implementata per validare il comportamento di ciascun metodo. Per facilitare la lettura, il codice è stato suddiviso isolando la configurazione iniziale e ogni singolo caso di test.

1. `IssueService.createIssue(String projectKey, IssueRequest request)` Configurazione dell'ambiente di Test

```
1 @ExtendWith(MockitoExtension.class)
2 class IssueServiceTest {
3
4     @Mock private IssueRepository issueRepository;
5     @Mock private ProjectRepository projectRepository;
6     @Mock private UserRepository userRepository;
7     @Mock private TagRepository tagRepository;
8     @Mock private AttachmentRepository attachmentRepository;
9     @Mock private ProjectService projectService;
10
11     @InjectMocks
12     private IssueService issueService;
13
14     private String projectKey;
15     private Project project;
16 }
```

```
17     @BeforeEach
18     void setUp() {
19         projectKey = "FRONT";
20         project = Project.builder()
21             .id(UUID.randomUUID())
22             .key(projectKey)
23             .name("BugBoard Core")
24             .description("test project")
25             .build();
26     }
```

Listing 6.1: Setup e definizione dei Mock

Casi di Test

```
1     @Test
2     void
3     createIssue_UsesDefaultStatusAndNoAssignee_WhenStatusAndTagsAreMissing
4     () {
5         IssueRequest request = createRequest(null, null, null);
6         when(projectRepository.findByKey(projectKey)).thenReturn(
7             Optional.of(project));
8         when(issueRepository.save(any(Issue.class))).thenReturn(
9             invocation -> invocation.getArgument(0));
10
11         Issue result = issueService.createIssue(projectKey, request);
12
13         assertNotNull(result);
14         assertEquals("Issue title", result.getTitle());
15         assertEquals("Issue description", result.getDescription());
16         assertEquals(project, result.getProject());
17         assertEquals(IssueStatus.TO_DO, result.getStatus());
18         assertEquals(IssuePriority.MEDIUM, result.getPriority());
19         assertEquals(IssueType.BUG, result.getType());
20         assertNull(result.getAssignee());
21         assertNotNull(result.getTags());
22         assertEquals(0, result.getTags().size());
23         verify(issueRepository).save(any(Issue.class));
24     }
```

Listing 6.2: Verifica dell'assegnazione dello status di default se mancante

```
1     @Test
2     void createIssue_PreservesExplicitStatus() {
3         IssueRequest request = createRequest(IssueStatus.IN_PROGRESS,
4             null, null);
5         when(projectRepository.findByKey(projectKey)).thenReturn(
6             Optional.of(project));
```

```
5         when(issueRepository.save(any(Issue.class))).thenAnswer(  
        invocation -> invocation.getArgument(0));  
6  
7         Issue result = issueService.createIssue(projectKey, request);  
8  
9         assertEquals(IssueStatus.IN_PROGRESS, result.getStatus());  
10        verify(issueRepository).save(any(Issue.class));  
11    }
```

Listing 6.3: Verifica del mantenimento dello status esplicito

```
1    @Test  
2    void createIssue_ThrowsWhenProjectDoesNotExist() {  
3        IssueRequest request = createRequest(null, null, null);  
4        when(projectRepository.findByKey(projectKey)).thenReturn(  
        Optional.empty());  
5  
6        ResourceNotFoundException exception = assertThrows(  
        ResourceNotFoundException.class, () -> issueService.createIssue(  
        projectKey, request));  
7  
8        assertEquals("Project not found with key: " + projectKey,  
        exception.getMessage());  
9        verify(issueRepository, never()).save(any(Issue.class));  
10    }
```

Listing 6.4: Gestione eccezione per progetto inesistente

```
1    @Test  
2    void createIssue_AssociatesConvertedTags() {  
3        UUID tagOneId = UUID.randomUUID();  
4        UUID tagTwoId = UUID.randomUUID();  
5  
6        TagResponse tagOne = TagResponse.builder()  
7            .id(tagOneId)  
8            .name("backend")  
9            .color("#112233")  
10           .projectKey(projectKey)  
11           .build();  
12        TagResponse tagTwo = TagResponse.builder()  
13            .id(tagTwoId)  
14            .name("frontend")  
15            .color("#445566")  
16            .projectKey(projectKey)  
17            .build();  
18  
19        IssueRequest request = createRequest(null, List.of(tagOne,  
        tagTwo), null);  
20        when(projectRepository.findByKey(projectKey)).thenReturn(  
        Optional.of(project));
```

```

21         when(issueRepository.save(any(Issue.class))).thenAnswer(
22             invocation -> invocation.getArgument(0));
23
24         Issue result = issueService.createIssue(projectKey, request);
25
26         assertNotNull(result.getTags());
27         assertEquals(2, result.getTags().size());
28
29         Tag firstTag = result.getTags().get(0);
30         Tag secondTag = result.getTags().get(1);
31
32         assertEquals(tagOneId, firstTag.getId());
33         assertEquals("backend", firstTag.getName());
34         assertEquals("#112233", firstTag.getColor());
35         assertEquals(project, firstTag.getProject());
36
37         assertEquals(tagTwoId, secondTag.getId());
38         assertEquals("frontend", secondTag.getName());
39         assertEquals("#445566", secondTag.getColor());
40         assertEquals(project, secondTag.getProject());
41     }

```

Listing 6.5: Verifica della corretta associazione dei Tag

```

1     @Test
2     void createIssue_AssignsIssueToRequestedProjectOnly() {
3         UUID otherProjectId = UUID.randomUUID();
4         Project otherProject = Project.builder()
5             .id(otherProjectId)
6             .name("Other Project")
7             .description("different project")
8             .build();
9
10        IssueRequest request = createRequest(null, null, null);
11        when(projectRepository.findByKey(projectKey)).thenReturn(
12            Optional.of(project));
13        when(issueRepository.save(any(Issue.class))).thenAnswer(
14            invocation -> invocation.getArgument(0));
15
16        Issue result = issueService.createIssue(projectKey, request);
17
18        assertEquals(project.getId(), result.getProject().getId());
19        assertEquals(project, result.getProject());
20        assertNotEquals(otherProjectId, result.getProject().getId());
21        assertNotEquals(otherProject, result.getProject());
22    }

```

Listing 6.6: Controllo sull'isolamento dell'Issue al solo progetto richiesto

```

1     @Test

```

```
2     void createIssue_IgnoresNullTagEntries() {
3         IssueRequest request = createRequest(null, Collections.
4         singletonList(null), null);
5         when(projectRepository.findByKey(projectKey)).thenReturn(
6         Optional.of(project));
7         when(issueRepository.save(any(Issue.class))).thenReturn(
8         invocation -> invocation.getArgument(0));
9
10        Issue result = issueService.createIssue(projectKey, request);
11
12        assertNotNull(result.getTags());
13        assertEquals(0, result.getTags().size());
14    }
```

Listing 6.7: Gestione di entry null all'interno lista dei Tag

```
1     @Test
2     void createIssue_LeavesTagsEmptyWhenListIsEmpty() {
3         IssueRequest request = createRequest(null, List.of(), null);
4         when(projectRepository.findByKey(projectKey)).thenReturn(
5         Optional.of(project));
6         when(issueRepository.save(any(Issue.class))).thenReturn(
7         invocation -> invocation.getArgument(0));
8
9         Issue result = issueService.createIssue(projectKey, request);
10
11        assertNotNull(result.getTags());
12        assertEquals(0, result.getTags().size());
13    }
```

Listing 6.8: Gestione di una lista di Tag vuota

```
1     @Test
2     void createIssue_LeavesAssigneeEmptyWhenUsernameIsNull() {
3         IssueRequest request = createRequest(null, null, null);
4         when(projectRepository.findByKey(projectKey)).thenReturn(
5         Optional.of(project));
6         when(issueRepository.save(any(Issue.class))).thenReturn(
7         invocation -> invocation.getArgument(0));
8
9         Issue result = issueService.createIssue(projectKey, request);
10
11        assertNull(result.getAssignee());
12    }
```

Listing 6.9: Gestione assegnatario mancante

```
1     @Test
2     void createIssue_AssignsAssigneeWhenUsernameIsProvided() {
```

```
3      User assignee = createUser("johndoe");
4      IssueRequest request = createRequest(null, null, assignee.
  getUsername());
5      when(projectRepository.findByKey(projectKey)).thenReturn(
Optional.of(project));
6      when(userRepository.findByUsername(assignee.getUsername())).
  thenReturn(Optional.of(assignee));
7      when(issueRepository.save(any(Issue.class))).thenAnswer(
  invocation -> invocation.getArgument(0));
8
9      Issue result = issueService.createIssue(projectKey, request);
10
11     assertNotNull(result.getAssignee());
12     assertEquals(assignee.getUsername(), result.getAssignee().
  getUsername());
13     assertEquals(assignee.getId(), result.getAssignee().getId());
14 }
```

Listing 6.10: Verifica assegnazione tramite username

```
1  @Test
2  void createIssue_SavesAttachmentsWhenProvided() {
3      AttachmentMetadataRequest meta1 = new AttachmentMetadataRequest
  ();
4      meta1.setOriginalFileName("report.pdf");
5      meta1.setBlobFileName("uuid-report.pdf");
6      meta1.setFileSize(2048L);
7      meta1.setExtension(".pdf");
8
9      AttachmentMetadataRequest meta2 = new AttachmentMetadataRequest
  ();
10     meta2.setOriginalFileName("screenshot.png");
11     meta2.setBlobFileName("uuid-screenshot.png");
12     meta2.setFileSize(4096L);
13     meta2.setExtension(".png");
14
15     IssueRequest request = createRequest(null, null, null);
16     request.setAttachments(List.of(meta1, meta2));
17
18     when(projectRepository.findByKey(projectKey)).thenReturn(
Optional.of(project));
19     when(issueRepository.save(any(Issue.class))).thenAnswer(
  invocation -> invocation.getArgument(0));
20     when(attachmentRepository.save(any(Attachment.class))).
  thenAnswer(invocation -> invocation.getArgument(0));
21
22     Issue result = issueService.createIssue(projectKey, request);
23
24     assertNotNull(result);
25     verify(attachmentRepository, times(2)).save(any(Attachment.class
```

```
26    ));  
    }
```

Listing 6.11: Verifica del salvataggio degli allegati presenti

```
1  @Test  
2  void createIssue_DoesNotSaveAttachmentsWhenListIsNull() {  
3      IssueRequest request = createRequest(null, null, null);  
4      request.setAttachments(null);  
5  
6      when(projectRepository.findByKey(projectKey)).thenReturn(  
Optional.of(project));  
7      when(issueRepository.save(any(Issue.class))).thenReturn(  
invocation -> invocation.getArgument(0));  
8  
9      issueService.createIssue(projectKey, request);  
10  
11     verify(attachmentRepository, never()).save(any(Attachment.class)  
);  
12 }
```

Listing 6.12: Assenza di allegati se lista allegati è null

```
1  @Test  
2  void createIssue_DoesNotSaveAttachmentsWhenListIsEmpty() {  
3      IssueRequest request = createRequest(null, null, null);  
4      request.setAttachments(List.of());  
5  
6      when(projectRepository.findByKey(projectKey)).thenReturn(  
Optional.of(project));  
7      when(issueRepository.save(any(Issue.class))).thenReturn(  
invocation -> invocation.getArgument(0));  
8  
9      issueService.createIssue(projectKey, request);  
10  
11     verify(attachmentRepository, never()).save(any(Attachment.class)  
);  
12 }  
13 }
```

Listing 6.13: Assenza di allegati se lista è vuota

2. AttachmentService.uploadAttachment(UUID issueId, MultipartFile file) Il metodo gestisce il caricamento e la validazione degli allegati multimediali. Riceve l'identificativo univoco della issue (`issueId`) e il file in ingresso (`file`). Verifica che il file non sia vuoto, genera il nome univoco, lo scrive sul filesystem temporaneo (`@TempDir`) e crea il record relazionale `Attachment`.

Configurazione dell'ambiente di Test

```
1 @ExtendWith(MockitoExtension.class)
2 class AttachmentServiceTest {
3
4     @Mock private AttachmentRepository attachmentRepository;
5     @Mock private IssueRepository issueRepository;
6     @InjectMocks private AttachmentService attachmentService;
7
8     private Issue dummyIssue;
9     private UUID issueId;
10
11     // JUnit 5 will automatically create this directory before tests and
12     // delete it afterward
13     @TempDir
14     Path tempUploadDir;
15
16     @BeforeEach
17     void setUp() {
18         issueId = UUID.randomUUID();
19         dummyIssue = Issue.builder()
20             .id(issueId)
21             .title("Test Issue")
22             .build();
23
24         // 1. Inject the temporary directory path into the service via
25         // reflection
26         ReflectionTestUtils.setField(attachmentService, "uploadDir",
27             tempUploadDir.toString());
28
29         // 2. Manually trigger the initialization method to simulate
30         // Spring's @PostConstruct
31         attachmentService.init();
32     }
33 }
```

Listing 6.14: Setup e definizione dei Mock per AttachmentService

Casi di Test

```
1 @Test
2 void uploadAttachment_ShouldThrowException_WhenFileIsEmpty() {
3     // Arrange
4     MultipartFile emptyFile = new MockMultipartFile("file", "test.
5     txt", "text/plain", new byte[0]);
6
7     when(issueRepository.findById(issueId)).thenReturn(Optional.of(
8     dummyIssue));
9
10    // Act & Assert
11 }
```

```
9      IllegalArgumentException exception = assertThrows(  
IllegalArgumentException.class, () -> {  
10          attachmentService.uploadAttachment(issueId, emptyFile);  
11      });  
12  
13      assertEquals("Cannot upload an empty file", exception.getMessage()  
());  
14      verify(attachmentRepository, never()).save(any(Attachment.class)  
);  
15  }
```

Listing 6.15: Gestione dell'eccezione per il caricamento di un file vuoto

```
1      @Test  
2      void  
uploadAttachment_ShouldThrowResourceNotFound_WhenIssueDoesNotExist()  
{  
3          // Arrange  
4          MultipartFile validFile = new MockMultipartFile("file", "test.  
txt", "text/plain", "Test Content".getBytes());  
5          when(issueRepository.findById(issueId)).thenReturn(Optional.  
empty());  
6  
7          // Act & Assert  
8          ResourceNotFoundException exception = assertThrows(  
ResourceNotFoundException.class, () -> {  
9              attachmentService.uploadAttachment(issueId, validFile);  
10          });  
11  
12          assertEquals(String.format("Issue with id %s not found", issueId  
) , exception.getMessage());  
13          verify(attachmentRepository, never()).save(any(Attachment.class)  
);  
14  }
```

Listing 6.16: Gestione dell'eccezione per l'aggiunta di un allegato a una Issue inesistente

```
1      @Test  
2      void  
uploadAttachment_ShouldSaveAndReturnAttachmentResponse_WhenValid() {  
3          // Arrange  
4          MultipartFile validFile = new MockMultipartFile("file", "test.  
png", "image/png", "Fake Image Content".getBytes());  
5          when(issueRepository.findById(issueId)).thenReturn(Optional.of(  
dummyIssue));  
6  
7          Attachment savedAttachment = new Attachment();  
8          savedAttachment.setId(UUID.randomUUID());  
9          savedAttachment.setFileName("test.png");  
10         savedAttachment.setIssue(dummyIssue);
```

```
11         when(attachmentRepository.save(any(Attachment.class))).  
12         thenReturn(savedAttachment);  
13  
14         // Act  
15         AttachmentResponse result = attachmentService.uploadAttachment(  
16         issueId, validFile);  
17  
18         // Assert  
19         assertNotNull(result);  
20         assertEquals("test.png", result.fileName());  
21  
22         verify(issueRepository, times(1)).findById(issueId);  
23         verify(attachmentRepository, times(1)).save(any(Attachment.class  
24         ));  
25     }
```

Listing 6.17: Verifica del salvataggio corretto e restituzione della risposta per un file valido

```
1     @Test  
2     void getAttachmentById_ShouldReturnAttachmentResponse_WhenFound() {  
3         // Arrange  
4         UUID attachmentId = UUID.randomUUID();  
5         Attachment attachment = new Attachment();  
6         attachment.setId(attachmentId);  
7         attachment.setFileName("document.pdf");  
8         attachment.setIssue(dummyIssue);  
9  
10        when(attachmentRepository.findById(attachmentId)).thenReturn(  
11        Optional.of(attachment));  
12  
13        // Act  
14        AttachmentResponse result = attachmentService.getAttachmentById(  
15        attachmentId);  
16  
17        // Assert  
18        assertNotNull(result);  
19        assertEquals("document.pdf", result.fileName());  
20        verify(attachmentRepository, times(1)).findById(attachmentId);  
21    }
```

Listing 6.18: Recupero corretto dei dettagli di un allegato tramite ID

```
1     @Test  
2     void getAttachmentById_ShouldThrowResourceNotFound_WhenNotFound() {  
3         // Arrange  
4         UUID attachmentId = UUID.randomUUID();  
5         when(attachmentRepository.findById(attachmentId)).thenReturn(  
6         Optional.empty());  
7     }
```

```

7      // Act & Assert
8      ResourceNotFoundException exception = assertThrows(
ResourceNotFoundException.class, () -> {
9          attachmentService.getAttachmentById(attachmentId);
10     });
11
12     assertEquals(String.format("Attachment with id %s not found",
attachmentId), exception.getMessage());
13 }

```

Listing 6.19: Gestione dell'eccezione per il recupero di un allegato inesistente

```

1  @Test
2  void getAttachmentsByIssueId_ShouldReturnListOfAttachmentResponses()
3  {
4      // Arrange
5      Attachment attachment = new Attachment();
6      attachment.setId(UUID.randomUUID());
7      attachment.setFileName("screen.png");
8      attachment.setIssue(dummyIssue);
9
10     when(attachmentRepository.findById(issueId)).thenReturn(
List.of(attachment));
11
12     // Act
13     List<AttachmentResponse> result = attachmentService.
getAttachmentsByIssueId(issueId);
14
15     // Assert
16     assertNotNull(result);
17     assertEquals(1, result.size());
18     assertEquals("screen.png", result.get(0).fileName());
19     verify(attachmentRepository, times(1)).findById(issueId);
20 }

```

Listing 6.20: Recupero della lista completa di allegati associati a una Issue

```

1  @Test
2  void uploadAttachment_ShouldHandleFilesWithoutExtension() {
3      // Arrange
4      MultipartFile fileWithoutExt = new MockMultipartFile("file", "
testfile", "text/plain", "Content".getBytes());
5      when(issueRepository.findById(issueId)).thenReturn(Optional.of(
dummyIssue));
6
7      Attachment savedAttachment = new Attachment();
8      savedAttachment.setId(UUID.randomUUID());
9      savedAttachment.setFileName("testfile");
10     savedAttachment.setFileExtension("");
11     savedAttachment.setIssue(dummyIssue);

```

```

12         when(attachmentRepository.save(any(Attachment.class))).
13         thenReturn(savedAttachment);
14
15         // Act
16         AttachmentResponse result = attachmentService.uploadAttachment(
17         issueId, fileWithoutExt);
18
19         // Assert
20         assertNotNull(result);
21         assertEquals("", result.fileExtension());
22         verify(attachmentRepository, times(1)).save(any(Attachment.class
23         ));
24     }

```

Listing 6.21: Gestione del salvataggio di un file privo di estensione

```

1     @Test
2     void uploadAttachment_ShouldThrowFileStorageException_OnIOException
3     () throws java.io.IOException {
4         // Arrange
5         MultipartFile badFile = mock(MultipartFile.class);
6         when(badFile.isEmpty()).thenReturn(false);
7         when(badFile.getOriginalFilename()).thenReturn("test.png");
8         when(badFile.getInputStream()).thenThrow(new java.io.IOException
9         ("Simulated IO Error"));
10
11         when(issueRepository.findById(issueId)).thenReturn(Optional.of(
12         dummyIssue));
13
14         // Act & Assert
15         FileStorageException exception = assertThrows(
16         FileStorageException.class, () -> {
17             attachmentService.uploadAttachment(issueId, badFile);
18         });
19
20         assertTrue(exception.getMessage().contains("Could not store file
21         test.png"));
22     }

```

Listing 6.22: Gestione dell'errore di I/O durante la scrittura sul file system

```

1     @Test
2     void uploadAttachment_ShouldHandleNullOriginalFileName() throws java
3     .io.IOException {
4         // Arrange
5         // We use Mockito to strictly force the return of a real 'null',
6         // bypassing MockMultipartFile logic
7         MultipartFile mockFile = mock(MultipartFile.class);
8         when(mockFile.isEmpty()).thenReturn(false);

```

```

7      when(mockFile.getOriginalFilename()).thenReturn(null); // Force
    real null!
8      when(mockFile.getSize()).thenReturn(10L);
9      // We must provide a valid InputStream because Files.copy will
    try to read from it
10     when(mockFile.getInputStream()).thenReturn(new java.io.
    ByteArrayInputStream("Content".getBytes()));
11
12     when(issueRepository.findById(issueId)).thenReturn(Optional.of(
    dummyIssue));
13
14     Attachment savedAttachment = new Attachment();
15     savedAttachment.setId(UUID.randomUUID());
16     savedAttachment.setFileName(null);
17     savedAttachment.setFileExtension("");
18     savedAttachment.setIssue(dummyIssue);
19
20     when(attachmentRepository.save(any(Attachment.class))).
    thenReturn(savedAttachment);
21
22     // Act
23     AttachmentResponse result = attachmentService.uploadAttachment(
    issueId, mockFile);
24
25     // Assert
26     assertNotNull(result);
27     assertEquals("", result.fileExtension());
28     assertNull(result.fileName());
29     verify(attachmentRepository, times(1)).save(any(Attachment.class
    ));
30 }

```

Listing 6.23: Gestione del salvataggio di un file con nome originale nullo

```

1  @Test
2  void init_ShouldThrowRuntimeException_OnIOException() throws java.io
    .IOException {
3      // Arrange
4      // Create an ACTUAL physical file (not a directory) inside the
    temporary folder
5      Path existingFile = java.nio.file.Files.createFile(tempUploadDir
    .resolve("dummyFile.txt"));
6
7      // Tell the AttachmentService to use this file path as the
    upload directory
8      ReflectionTestUtils.setField(attachmentService, "uploadDir",
    existingFile.toString());
9
10     // Act & Assert
11     // Attempting to create a directory over an existing physical

```

```
file will trigger an IOException
12     RuntimeException exception = assertThrows(RuntimeException.class
13     , () -> {
14         attachmentService.init();
15     });
16     assertEquals("Could not create upload directory!", exception.
17     getMessage());
18 }
```

Listing 6.24: Gestione dell'errore durante l'inizializzazione della cartella di upload

```
1     @Test
2     void loadFileAsResource_Success() throws java.io.IOException {
3         UUID attachmentId = UUID.randomUUID();
4         Path testFile = tempUploadDir.resolve("test-download.txt");
5         java.nio.file.Files.writeString(testFile, "Hello World");
6
7         Attachment attachment = Attachment.builder()
8             .id(attachmentId)
9             .fileName("test-download.txt")
10            .filePath(testFile.toString())
11            .build();
12        when(attachmentRepository.findById(attachmentId)).thenReturn(
Optional.of(attachment));
13
14        org.springframework.core.io.Resource resource =
attachmentService.loadFileAsResource(attachmentId);
15
16        assertNotNull(resource);
17        assertTrue(resource.exists());
18    }
```

Listing 6.25: Caricamento corretto di un file come risorsa per il download

```
1     @Test
2     void loadFileAsResource_ThrowsWhenFileNotFound() {
3         UUID attachmentId = UUID.randomUUID();
4         Attachment attachment = Attachment.builder()
5             .id(attachmentId)
6             .fileName("not-exists.txt")
7             .filePath(tempUploadDir.resolve("not-exists.txt").
toString())
8             .build();
9         when(attachmentRepository.findById(attachmentId)).thenReturn(
Optional.of(attachment));
10
11        assertThrows(ResourceNotFoundException.class, () ->
attachmentService.loadFileAsResource(attachmentId));
12    }
```

Listing 6.26: Gestione dell'eccezione in caso di file fisico non trovato

```

1      @Test
2      void loadFileAsResource_ThrowsWhenMalformedURLException() {
3          UUID attachmentId = UUID.randomUUID();
4          Attachment attachment = Attachment.builder()
5              .id(attachmentId)
6              .fileName("bad.txt")
7              .filePath("\u0000://invalid")
8              .build();
9          when(attachmentRepository.findById(attachmentId)).thenReturn(
Optional.of(attachment));
10
11          assertThrows(java.nio.file.InvalidPathException.class, () ->
attachmentService.loadFileAsResource(attachmentId));
12      }
13  }

```

Listing 6.27: Gestione dell'eccezione dovuta a un percorso di sistema malformato

3. ProjectService.addMembersToProject(UUID projectId, List<UUID> userIdsToAdd)

Il metodo consente di aggiungere un elenco di utenti (`userIdsToAdd`) all'interno del team di un determinato progetto (`projectId`). Viene verificata l'autenticazione dell'utente (`SecurityContextHolder`), l'happy path di aggiunta di diversi membri al progetto, l'esclusione di utenti inesistenti o già parte nel progetto.

Configurazione dell'ambiente di Test

```

1  @ExtendWith(MockitoExtension.class)
2  class ProjectMemberServiceTest {
3
4      @Mock private ProjectRepository projectRepository;
5      @Mock private UserRepository userRepository;
6      @Mock private SecurityContext securityContext;
7      @Mock private Authentication authentication;
8
9      @InjectMocks private ProjectService projectService;
10
11      private User adminUser;
12      private User technicalUser1;
13      private User technicalUser2;
14      private Project testProject;
15      private UUID projectId;
16      private UUID adminId;
17  }

```



```
18     @BeforeEach
19     void setUp() {
20         adminId = UUID.randomUUID();
21         UUID techId1 = UUID.randomUUID();
22         UUID techId2 = UUID.randomUUID();
23         projectId = UUID.randomUUID();
24
25         adminUser = User.builder()
26             .id(adminId)
27             .username("admin")
28             .email("admin@example.com")
29             .role(Role.ADMIN)
30             .build();
31
32         technicalUser1 = User.builder()
33             .id(techId1)
34             .username("tech1")
35             .email("tech1@example.com")
36             .role(Role.technical)
37             .build();
38
39         technicalUser2 = User.builder()
40             .id(techId2)
41             .username("tech2")
42             .email("tech2@example.com")
43             .role(Role.technical)
44             .build();
45
46         testProject = Project.builder()
47             .id(projectId)
48             .name("Test Project")
49             .description("Test")
50             .members(new ArrayList<>(List.of(adminUser,
technicalUser1)))
51             .build();
52     }
53
54     private void setupSecurityContext() {
55         SecurityContextHolder.setContext(securityContext);
56         when(securityContext.getAuthentication()).thenReturn(
authentication);
57         when(authentication.isAuthenticated()).thenReturn(true);
58         when(authentication.getName()).thenReturn(adminId.toString());
59     }
```

Listing 6.28: Setup e definizione dei Mock per ProjectMemberService

Casi di Test

```

1      @Test
2      void testAddMembersToProject_OnlyAdminCanAdd() {
3          setupSecurityContext();
4          User technicalUser = User.builder()
5              .id(UUID.randomUUID())
6              .username("tech")
7              .email("tech@example.com")
8              .role(Role.TECHNICAL)
9              .build();
10
11          when(authentication.getName()).thenReturn(technicalUser.getId().toString());
12          when(userRepository.findById(technicalUser.getId())).thenReturn(Optional.of(technicalUser));
13
14          assertThrows(UnauthorizedException.class, () ->
15              projectService.addMembersToProject(projectId, List.of(technicalUser2.getId()))
16          );
17      }

```

Listing 6.29: Verifica che solo un utente ADMIN possa aggiungere membri

```

1      @Test
2      void testAddMembersToProject_Success() {
3          setupSecurityContext();
4          List<UUID> userIdsToAdd = List.of(technicalUser2.getId());
5
6          when(projectRepository.findById(projectId)).thenReturn(Optional.of(testProject));
7          when(userRepository.findById(adminId)).thenReturn(Optional.of(adminUser));
8          when(userRepository.findAllById(userIdsToAdd)).thenReturn(List.of(technicalUser2));
9          when(projectRepository.save(any(Project.class))).thenReturn(testProject);
10
11          List<User> result = projectService.addMembersToProject(projectId, userIdsToAdd);
12
13          assertNotNull(result);
14          assertEquals(1, result.size());
15          assertEquals(technicalUser2.getId(), result.get(0).getId());
16          verify(projectRepository, times(1)).save(any(Project.class));
17      }

```

Listing 6.30: Verifica dell'aggiunta corretta dei membri al progetto

```

1      @Test
2      void testAddMembersToProject_ExcludesDuplicates() {

```

```

3      setupSecurityContext();
4      List<UUID> userIdsToAdd = List.of(technicalUser1.getId(),
technicalUser2.getId());
5
6      when(projectRepository.findById(projectId)).thenReturn(Optional.
of(testProject));
7      when(userRepository.findById(adminId)).thenReturn(Optional.of(
adminUser));
8      when(userRepository.findAllById(userIdsToAdd)).thenReturn(List.
of(technicalUser1, technicalUser2));
9      when(projectRepository.save(any(Project.class))).thenReturn(
testProject);
10
11     List<User> result = projectService.addMembersToProject(projectId
, userIdsToAdd);
12
13     assertNotNull(result);
14     assertEquals(1, result.size());
15     assertEquals(technicalUser2.getId(), result.get(0).getId());
16 }

```

Listing 6.31: Verifica dell'esclusione di utenti già presenti nel progetto (duplicati)

```

1     @Test
2     void testAddMembersToProject_UserNotFound() {
3         setupSecurityContext();
4         List<UUID> userIdsToAdd = List.of(UUID.randomUUID());
5
6         when(projectRepository.findById(projectId)).thenReturn(Optional.
of(testProject));
7         when(userRepository.findById(adminId)).thenReturn(Optional.of(
adminUser));
8         when(userRepository.findAllById(userIdsToAdd)).thenReturn(List.
of());
9
10        assertThrows(ResourceNotFoundException.class, () ->
11            projectService.addMembersToProject(projectId,
userIdsToAdd)
12        );
13    }

```

Listing 6.32: Gestione dell'eccezione in caso di utenti inesistenti

```

1     @Test
2     void testAddMembersToProject_RejectsAdminUsers() {
3         setupSecurityContext();
4         UUID anotherAdminId = UUID.randomUUID();
5         User anotherAdmin = User.builder()
6             .id(anotherAdminId)
7             .username("admin2")

```

```
8         .email("admin2@example.com")
9         .role(Role.ADMIN)
10        .build();
11
12        when(projectRepository.findById(projectId)).thenReturn(Optional.
of(testProject));
13        when(userRepository.findById(adminId)).thenReturn(Optional.of(
adminUser));
14        when(userRepository.findAllById(List.of(anotherAdminId))).
thenReturn(List.of(anotherAdmin));
15
16        assertThrows(UnauthorizedException.class, () ->
17            projectService.addMembersToProject(projectId, List.of(
anotherAdminId))
18        );
19    }
```

Listing 6.33: Verifica del divieto di aggiungere altri utenti con ruolo ADMIN

```
1    @Test
2    void testAddMembersToProject_AllAlreadyMembers() {
3        setupSecurityContext();
4        List<UUID> userIdsToAdd = List.of(technicalUser1.getId());
5
6        when(projectRepository.findById(projectId)).thenReturn(Optional.
of(testProject));
7        when(userRepository.findById(adminId)).thenReturn(Optional.of(
adminUser));
8        when(userRepository.findAllById(userIdsToAdd)).thenReturn(List.
of(technicalUser1));
9
10       assertThrows(IllegalArgumentException.class, () ->
11           projectService.addMembersToProject(projectId,
12       userIdsToAdd)
13       );
14   }
```

Listing 6.34: Gestione dell'eccezione se tutti gli utenti passati sono già membri

6.4 Test Plan della creazione di una Issue

Il presente piano di test definisce le verifiche applicate al metodo `IssueService.createIssue(String projectKey, IssueRequest request)`. L'analisi è circoscritta al comportamento del Service Layer e alle interazioni con le dipendenze simulate mediante Mockito.

6.4.1 Obiettivi

L'obiettivo principale del piano di test è verificare che il metodo `createIssue` costruisca e persista correttamente una nuova issue, rispettando le regole applicative previste.

In particolare, i test verificano che:

- la issue venga associata esclusivamente al progetto identificato da `projectKey`;
- titolo, descrizione, priorità e tipologia siano trasferiti correttamente dalla richiesta all'entità `Issue`;
- in assenza di uno stato esplicito venga utilizzato lo stato predefinito `IssueStatus.TO_DO`;
- uno stato esplicitamente specificato nella richiesta venga mantenuto;
- l'assegnatario rimanga assente quando non viene fornito alcuno username;
- un utente esistente venga correttamente associato alla issue quando viene specificato il relativo username;
- i tag presenti nella richiesta vengano convertiti in entità `Tag` e associati al progetto della issue;
- collezioni di tag assenti, vuote o contenenti elementi nulli siano gestite senza produrre associazioni non valide;
- i metadati degli allegati vengano persistiti solamente quando effettivamente presenti;
- la creazione venga interrotta quando la chiave fornita non identifica alcun progetto;
- in caso di errore non venga eseguita alcuna operazione di persistenza della issue.

6.4.2 Scope

Il piano di test comprende esclusivamente il metodo `IssueService.createIssue` e le regole di dominio direttamente applicate durante la creazione.

Rientrano nello scope:

- la ricerca del progetto attraverso `ProjectRepository`;
- la costruzione dell'entità `Issue`;
- l'applicazione dello stato predefinito;
- l'associazione dell'eventuale assegnatario;

- la conversione e l'associazione dei tag;
- la persistenza della issue mediante `IssueRepository`;
- la persistenza dei metadati degli allegati mediante `AttachmentRepository`;
- la gestione del progetto inesistente;
- la verifica delle interazioni con i repository simulati.

Non rientrano nello scope:

- le operazioni di aggiornamento o eliminazione di una issue;
- le modifiche dello stato successive alla creazione;
- l'aggiunta o la rimozione di tag successive alla creazione;
- il trasferimento del contenuto binario degli allegati;
- l'integrazione con un database reale;
- la verifica delle transazioni in un ambiente di persistenza reale;
- la validazione delle annotazioni Bean Validation, poiché i test invocano direttamente il Service Layer assumendo una `IssueRequest` già valida.

6.4.3 Precondizioni

L'esecuzione dei casi di test assume le seguenti precondizioni:

- le dipendenze del service sono sostituite da mock Mockito;
- ogni test è indipendente dagli altri e non condivide stato persistente;
- la richiesta contiene un titolo e una descrizione non vuoti;
- i valori di priorità e tipologia appartengono ai rispettivi domini enumerati;
- il metodo `IssueRepository.save` restituisce la medesima entità ricevuta, simulandone la corretta persistenza;
- quando richiesto dallo scenario, il progetto e l'assegnatario sono restituiti dai rispettivi repository.

6.4.4 Classi di equivalenza

Il dominio degli ingressi è stato suddiviso nelle seguenti classi di equivalenza. Ogni classe raggruppa valori che determinano il medesimo comportamento osservabile del metodo.

Chiave del progetto

CE-P1 – Progetto esistente: `projectKey` identifica un progetto presente nel repository. La issue viene creata, associata al progetto restituito e persistita.

CE-P2 – Progetto inesistente: `projectKey` non identifica alcun progetto. Il metodo solleva `ResourceNotFoundException` e la issue non viene persistita.

Dati principali della issue

CE-D1 – Dati validi: titolo e descrizione sono valorizzati, mentre priorità e tipologia appartengono ai rispettivi valori enumerati. I dati vengono trasferiti senza modifiche dalla richiesta alla nuova entità.

Stato iniziale

CE-S1 – Stato assente: il campo `status` è `null`. Alla issue viene assegnato automaticamente lo stato `IssueStatus.TO_DO`.

CE-S2 – Stato esplicito: il campo `status` contiene un valore valido. Il valore fornito viene mantenuto nella issue creata.

Tag

CE-T1 – Collezione assente: il campo `tags` è `null`. La issue viene creata con una collezione di tag non nulla e vuota.

CE-T2 – Collezione vuota: il campo `tags` è una lista vuota. Non viene creato alcun tag e la issue contiene una collezione vuota.

CE-T3 – Tag validi: la richiesta contiene uno o più oggetti `TagResponse` validi. Per ciascun elemento viene costruita un'entità `Tag`, conservando identificativo, nome e colore e associandola al progetto della issue.

CE-T4 – Elementi nulli: la collezione contiene elementi `null`. Tali elementi vengono ignorati e non producono associazioni nella issue.

Assegnatario

CE-A1 – Username assente: `assigneeUsername` è `null`. Non viene effettuata alcuna associazione e il campo `assignee` della `issue` rimane `null`.

CE-A2 – Utente esistente: `assigneeUsername` identifica un utente restituito da `UserRepository`. L'utente viene associato alla nuova `issue`, conservandone identificativo e `username`.

Metadati degli allegati

CE-AT1 – Collezione assente: il campo `attachments` è `null`. Non viene invocato `AttachmentRepository.save`.

CE-AT2 – Collezione vuota: il campo `attachments` contiene una lista vuota. Non viene persistito alcun metadato.

CE-AT3 – Metadati presenti: la richiesta contiene uno o più oggetti `AttachmentMetadataRequest`. Per ogni elemento viene costruita e persistita un'entità `Attachment` associata alla `issue` salvata.

6.4.5 Tracciabilità dei casi di test

Le classi di equivalenza sono coperte dai casi di test automatici nel modo seguente:

- **CE-P1, CE-D1, CE-S1, CE-T1 e CE-A1:** `createIssue_UsesDefaultStatusAndNoAssignee_WhenStatusAndTagsAreMissing`;
- **CE-S2:** `createIssue_PreservesExplicitStatus`;
- **CE-P2:** `createIssue_ThrowsWhenProjectDoesNotExist`;
- **CE-T3:** `createIssue_AssociatesConvertedTags`;
- **CE-P1:** `createIssue_AssignsIssueToRequestedProjectOnly`;
- **CE-T4:** `createIssue_IgnoresNullTagEntries`;
- **CE-T2:** `createIssue_LeavesTagsEmptyWhenListIsEmpty`;
- **CE-A1:** `createIssue_LeavesAssigneeEmptyWhenUsernameIsNull`;
- **CE-A2:** `createIssue_AssignsAssigneeWhenUsernameIsProvided`;
- **CE-AT3:** `createIssue_SavesAttachmentsWhenProvided`;
- **CE-AT1:** `createIssue_DoesNotSaveAttachmentsWhenListIsNull`;
- **CE-AT2:** `createIssue_DoesNotSaveAttachmentsWhenListIsEmpty`.

6.4.6 Criteri di accettazione

La funzionalità di creazione di una issue è considerata conforme quando sono soddisfatti tutti i seguenti criteri:

1. con una chiave di progetto valida, la issue viene associata esattamente al progetto restituito da `ProjectRepository`;
2. titolo, descrizione, priorità e tipologia della issue coincidono con i valori presenti nella richiesta;
3. se lo stato non è specificato, la issue viene creata nello stato `TO_DO`;
4. se lo stato è specificato, il valore fornito non viene sostituito dal valore predefinito;
5. se l'assegnatario non è indicato, la issue viene creata senza assegnatario;
6. se viene indicato un utente esistente, la issue viene associata esattamente a tale utente;
7. una collezione di tag assente o vuota produce una collezione di tag non nulla e vuota;
8. gli elementi nulli presenti nella collezione dei tag vengono ignorati;
9. ciascun tag valido conserva identificativo, nome e colore ed è associato al progetto della issue;
10. una collezione di allegati assente o vuota non determina alcuna invocazione di `AttachmentRepository.save`;
11. per una collezione contenente n metadati validi vengono effettuate esattamente n operazioni di persistenza degli allegati;
12. se il progetto non esiste, viene sollevata `ResourceNotFoundException` con il messaggio atteso;
13. in caso di progetto inesistente, `IssueRepository.save` non viene mai invocato;
14. tutti i casi di test risultano superati in maniera deterministica e indipendente dallo stato di risorse esterne.

7 Glossario

A

Admin utente con privilegi di gestione, in grado di creare e assegnare utenti e issue.

Affidabilità misura della capacità del sistema di funzionare in modo corretto e continuativo nel tempo.

Angular framework JavaScript per lo sviluppo di applicazioni web single-page.

Architettura struttura logica e fisica che definisce l'organizzazione dei componenti software.

Azure Blob Storage servizio di storage cloud di Microsoft usato da BugBoard26 per salvare gli allegati (immagini, file) associati alle issue.

B

Backend parte dell'applicazione che gestisce la logica, il database e la comunicazione tra client e server.

Background tecnico insieme delle conoscenze, competenze e tecnologie pregresse che costituiscono la base teorica e pratica di un team di sviluppo.

Boolean tipo di dato che può assumere solo due valori: vero o falso.

Bug errore o comportamento anomalo nel software che causa un malfunzionamento.

Bootstrap framework CSS (v5.x) adottato nel frontend per realizzare interfacce responsive, griglie e componenti stilizzati.

C

Casi d'uso rappresentazione delle interazioni tra un attore e il sistema per raggiungere un obiettivo specifico.

Chrome browser web sviluppato da Google.

Click azione dell'utente che interagisce con un elemento grafico.

Cloud computing modello di erogazione di servizi informatici tramite Internet.

Code quality (Qualità del codice) grado di leggibilità, modularità e manutenibilità del codice sorgente.

Commenti annotazioni nel codice che descrivono funzionalità o scelte implementative.

Concorrenti utenti o processi che operano simultaneamente sul sistema.

Containerizzato modalità di distribuzione del software all'interno di contenitori isolati, ad esempio tramite Docker.

CSRF vulnerabilità che consente a un utente malintenzionato di eseguire azioni non autorizzate su un sito autenticato.

CI/CD pratica di ****Continuous Integration / Continuous Delivery (o Deployment)**** che automatizza compilazione, test e rilascio del software; nel progetto è implementata con GitHub Actions.

D

Deadline scadenza entro la quale una determinata attività o task deve essere completata.

Degrado peggioramento delle prestazioni del sistema dovuto a carichi elevati.

Dipendenze librerie o componenti esterni necessari al corretto funzionamento del software.

Docker piattaforma per creare, distribuire ed eseguire container software.

HikariCP pool di connessioni JDBC ad alte prestazioni configurato in Spring Boot (vedi `'spring.datasource.hikari.*'` in `'application.properties'`).

E

Edge browser web sviluppato da Microsoft.

Efficienza misura delle risorse necessarie per eseguire operazioni (tempo, memoria, CPU).

Esportare operazione per convertire o salvare dati in formati condivisibili (CSV, PDF, ecc.).

Etichette parole chiave che classificano o raggruppano issue e bug.

F

File entità che rappresenta un'informazione salvata in memoria.

File explorer finestra di sistema usata per cercare e selezionare file locali.

Firefox browser open source sviluppato da Mozilla.

Flusso sequenza ordinata di azioni o eventi.

Formalismo insieme delle regole che definiscono il modo corretto di rappresentare i casi d'uso.

Framework struttura software riutilizzabile che facilita lo sviluppo applicativo.

Frontend parte dell'applicazione visibile e interattiva con l'utente.

Funzionalità core insieme delle funzioni principali del sistema.

G

Gerarchia relazione di subordinazione o dipendenza tra elementi o ruoli.

Git sistema di controllo versione distribuito.

H

Hashing tecnica per trasformare dati (es. password) in una stringa di lunghezza fissa non invertibile.

HEX / RRGGBB codifica esadecimale utilizzata per rappresentare i colori.

Hibernate framework Java per la gestione della persistenza dei dati.

I

Icona elemento grafico che rappresenta un'azione o funzione.

IDE ambiente di sviluppo integrato che facilita la scrittura e il debug del codice.

Identificativo codice univoco che distingue un elemento nel sistema.

Image file grafico o rappresentazione visiva associata a un'issue.

Interfaccia insieme degli elementi grafici e funzionali con cui l'utente interagisce.

Interazioni comunicazioni o scambi di informazioni tra attori e sistema.

Issue registrazione di un problema, richiesta o attività da gestire nel sistema.

ISO/IEC25010 standard internazionale per la valutazione della qualità del software (8 caratteristiche: funzionalità, affidabilità, usabilità, efficienza, manutenibilità, portabilità, sicurezza, compatibilità).

IEEE830-1998 standard che definisce le linee guida per la redazione di Specifiche dei Requisiti Software (SRS).

UML Unified Modeling Language, linguaggio grafico standard per la modellazione di sistemi software (use-case diagram, class diagram, sequence diagram, ...).

J

Jakarta EE piattaforma Java per lo sviluppo di applicazioni enterprise.

Java linguaggio di programmazione orientato agli oggetti.

JavaDoc sistema di documentazione automatica per codice Java.

JSON Web Token (JWT) standard aperto per la trasmissione sicura di informazioni tra client e server.

JS Doc strumento per generare documentazione dal codice JavaScript.

L

Lint strumento che analizza il codice per rilevare errori sintattici o stilistici.

Linguaggio di programmazione mezzo formale per scrivere istruzioni comprensibili da un computer.

Login procedura di autenticazione che consente l'accesso al sistema.

Log strutturato registro di eventi con formato standard che facilita l'analisi automatizzata.

M

Manutenibilità facilità con cui il software può essere corretto o migliorato nel tempo.

MB unità di misura della quantità di dati (megabyte).

Menù elenco di opzioni tra cui l'utente può scegliere.

Messaggio di errore testo mostrato quando si verifica un problema nell'esecuzione.

Metodologia insieme di processi e strumenti adottati nello sviluppo software.

Metodologie di sviluppo approcci sistematici come Agile, Waterfall.

Modale finestra sovrapposta che richiede l'interazione dell'utente.

Modello di Cockburn formalismo per la descrizione strutturata dei casi d'uso.

Modifiche strutturali cambiamenti significativi nell'architettura o nella base di codice.

O

Requisiti operativi requisiti che definiscono le condizioni di utilizzo e manutenzione del sistema.

U

UI-(User Interface) interfaccia grafica con cui l'utente interagisce.

URL indirizzo che identifica una risorsa in rete.

URL param parametro aggiunto a un URL per trasmettere informazioni.

Usabilità facilità con cui l'utente può utilizzare un sistema.

Up time tempo in cui il sistema risulta operativo e accessibile.

V

Validazione processo con cui si verifica che un sistema, prodotto o software soddisfi realmente i bisogni e le aspettative dell'utente finale.

Version Control sistema che gestisce e traccia le modifiche apportate ai file di un progetto nel tempo.

X

XSS vulnerabilità nelle applicazioni web che permette a un attaccante di inserire ed eseguire script non autorizzati nel browser di un altro utente.

Riferimenti Bibliografici

- [1] A. Cockburn, *Writing Effective Use Cases*. Addison-Wesley Professional, 2001. DOI: 10.1145/505894.505918. indirizzo: <https://doi.org/10.1145/505894.505918>.
- [2] Y. W. Bingyang Wei Lin Deng, “A Practical Style Guide and Templates Repository for Writing Effective Use Cases,” *Software Engineering Research, Management and Applications (SERA 2022)*, pp. 67–80, 2022. DOI: 10.1007/978-3-031-09145-2_5.
- [3] B. Wei. “Use Case Style Guide and Templates Repository.” Repository GitHub, GitHub. indirizzo: <https://github.com/Washingtonwei/use-case-style-guide>.
- [4] S. Parente Martone. “UC1- Creazione issue, prototipo mock-up. ”indirizzo: <https://www.figma.com/proto/w40cAPfZoJtGVIUNTzSeXF/BugBoard26?node-id=39-36&t=mPQpbtb5aucC9n5a-1>.
- [5] M. Pollio. “UC2- Creazione etichette, prototipo mock-up. ”indirizzo: <https://www.figma.com/proto/w40cAPfZoJtGVIUNTzSeXF/BugBoard26?node-id=108-609&t=n60xW5kdKw7LNRI7-1>.
- [6] M. Penna. “UC3 - Ordinamento, prototipo mock-up. ”indirizzo: <https://www.figma.com/proto/w40cAPfZoJtGVIUNTzSeXF/BugBoard26?node-id=24-13&t=mPQpbtb5aucC9n5a-1>.
- [7] M. Li. “Java Statistics: Adoption, Usage, and Future Trends. ”indirizzo: <https://www.seconddtalent.com/resources/domain-java-statistics/>.